CrashScope LLM: Automatically Discovering, Reporting and Reproducing Android Application Crashes

Software Requirement Specification

SE 801 – Software Project Lab III

Submitted By:

Arnab Banik

BSSE 1230

Supervised By:

Dr. Adwait Nadkarni
Associate Professor
Computer Science
William & Mary

Internal Supervisor:

Dr. Md. Shariful Islam
Professor
Institute of Information Technology
University of Dhaka



Internal Supervisor's Signature:_____

Letter of Transmittal

1st December 2024

BSSE 4th Year Exam Committee

Institute of Information Technology

University of Dhaka

Subject: Technical Report Submission – **Test Case Generation Using CrashScope with LLMs**

Dear Sir,

I am submitting the technical report for my Software Project Lab 3, "Test Case Generation Using CrashScope with LLMs". This report comprehensively covers the project's technical details, including design, implementation, and testing phases. While I've tried to do my best, I welcome your feedback for improvement.

Thank you for your consideration.

Sincerely,

Arnab Banik

BSSE 1230

Institute of Information Technology

University of Dhaka

Acknowledgment

I am grateful to the Institute of Information Technology, and SPL lab of William & Mary for giving me such a tremendous opportunity to work on “CrashScope LLM”. I would like to convey my tremendous gratitude to my supervisor, Dr. Md. Shariful Islam, Professor, Institute of Information Technology, University of Dhaka, for providing me guidelines about how I can prepare this analysis document. I would also like to thank Dr. Adwait Nadkarni, Class of 1953 Associate Professor, Computer Science, William & Mary for managing and overseeing the progress of the project. Lastly, I would like to thank my classmates. They have always been helpful and provided valuable insights from time to time.

Abstract

Automated testing is a critical aspect of Android application development, ensuring robustness and stability before deployment. CrashScope, an existing tool, automates this process by dynamically interacting with an application's UI to identify and document crashes. It provides comprehensive reports, including stack traces, reproduction steps, and replayable scripts, without requiring instrumentation of the application or Android OS. However, its current exploration strategies—top-down, bottom-up, and random—lack contextual awareness, potentially limiting its effectiveness.

This project aims to enhance CrashScope by integrating large language models (LLMs) to enable intelligent UI exploration and advanced crash report generation. The proposed LLM-based strategy will utilize contextual information, such as input labels and descriptions, to infer expected behaviors and simulate user interactions. Additionally, the LLM will analyze crash stack traces to produce concise, human-readable reports, offering developers actionable insights into crash causes.

By combining CrashScope's existing features with the capabilities of LLMs, this project seeks to streamline the testing process, improve crash detection accuracy, and provide developers with a powerful tool to enhance the quality and reliability of their Android applications.

Table of Contents

CrashScope LLM: Automatically Discovering, Reporting and Reproducing Android Application Crashes....1
Letter of Transmittal....2
Acknowledgment....3
Abstract....4
1 Introduction....8
1.1 Purpose....8
1.2 Scope....8
1.3 Assumptions....8
2 Project Description....9
2.1 Quality Function Deployment....9
2.1.1 Normal Requirements....9
2.1.2 Expected Requirements....9
2.1.3 Exciting Requirements....9
3 Usage Scenario....10
4 Scenario Based Modeling....11
4.1 Use Case Diagram....11
4.1.1 Use Case Level 0 – CrashScope LLM....11
4.1.2 Use Case Level 1 – Modules of CrashScope LLM....12
4.1.2.1 Apk Runner....12
4.1.2.2 UI Feature Extractor....13
4.1.2.3 Input Sequence Predictor....13
4.1.2.4 Generator....13
4.1.2.5 Result Display....13
4.1.2.6 Script Runner....13
4.1.3 Use Case Level 1.1 – Generator....14
5 Class-Based Modeling....15
5.1 Analysis Classes....15
5.2 Class Cards....16
5.3 CRC Diagram....19
6 Architectural Design....20
6.1 Architectural Context Diagram....20
6.2 Top Level Components....21
6.3 Instantiation of the Architecture....22
6.3.1 Explorer....22
6.3.2 Script Runner....23
7 Preliminary Test Plan....24
7.1 High Level Description of Testing Goals....24
7.2 Test Cases....24
8 Conclusion....28
9 References....29

Table of Figures

Figure 1: Use Case Level 0.....11
Figure 2: Use Case Level 1 - Modules of CrashScope LLM.....12
Figure 3: Use Case Level 1.1 - Generator.....14
Figure 4: CRC Diagram.....19
Figure 5: Architectural Context Diagram.....20
Figure 6: Top Level Components.....21
Figure 7: Layered Architecture for the Explorer Component.....22
Figure 8: Layered Architecture for the Script Runner Component.....23

Index of Tables

Table 1: Class Card - CrashScope....16
Table 2: Class Card - ApkRunner....16
Table 3: Class Card - Script Runner....16
Table 4: Class Card – Android Emulator....17
Table 5: Class Card – Feature Extractor....17
Table 6: Class Card – Report Generator....17
Table 7: Class Card – Test Case Generator....18
Table 8: Class Card – LLM....18
Table 9: Class Card - Script Runner....18

1 Introduction

CrashScope is an automated testing tool for Android apps that works without modifying the app or Android OS. It only requires the APK file to begin testing. CrashScope detects crashes during app usage by systematically exploring the app and generating inputs like taps, text entries, and sensor interactions to trigger crashes. When a crash occurs, it provides detailed reports. LLMs can enhance CrashScope by generating realistic test cases and improving the readability of crash reports and stack traces.

1.1 Purpose

The purposes of this document are-

- Identify and analyze the requirements
- Design the architecture
- Design the test plan
- Describe methodology
- Reduce the development effort
- Improve understanding

1.2 Scope

The scope of the project is given below-

- Desktop tool
- Test only .apk files
- Explore the android UI of the app
- Generate reports

1.3 Assumptions

The assumptions of the project are-

- The source code will not have any compilation errors
- The source code will contain a main function

2 Project Description

For this project, I have identified the developer community as the stakeholders of this project. Thus I have conducted several interviews with some of the developers currently working in the industry to gain insight into their views on the issue of Android testing tools. After these discussions, I have determined some preliminary set of solution requirements for this project.

This chapter presents the Quality function deployment and the usage scenario of the tool.

2.1 Quality Function Deployment

Quality Function Deployment (QFD) is a technique that translates the needs of the customer into technical requirements for the software. With respect to this project, the following requirements are identified-

2.1.1 Normal Requirements

NR1. Testing Android apps using CrashScope without instrumentation.

NR2. Generating crash reports with stack traces, screenshots, and reproduction steps.

NR3. Supporting existing exploration strategies: Top-down, Bottom-up, and Random.

NR4. Providing a transition graph for app navigation paths.

NR5. Functional and reproducible replayable scripts for crash scenarios.

NR6. Integration of LLM to explore UI elements intelligently.

NR7. LLM-enhanced test reports.

2.1.2 Expected Requirements

ER1. Configurable UI exploration strategies using LLM alongside existing approaches

ER2. A user friendly and informative interface.

2.1.3 Exciting Requirements

EXR1. Context-driven decision-making by the LLM for intelligent and efficient testing.

EXR2. Real-time updates during testing through the web application.

3 Usage Scenario

The user, an Android app developer, has built a new application and wants to ensure it is free from crashes before releasing it to the public. The CrashScope LLM tool offers an intelligent and automated way to test Android apps by simply uploading the `.apk` file, without needing to modify the app or the Android OS.

UI Exploration Strategy: The user has to upload the .apk upon opening the app. After the upload completes, the system analyzes the file for validity and prepares it for testing. The user is then presented with options to configure how the tool should explore the app. These options include traditional exploration strategies such as top-down, bottom-up, and random traversal. However, the user is particularly intrigued by the new LLM-based exploration feature and decides to enable it.

Exploration: The user configures the parameters for testing, such as setting time limits or excluding certain UI elements from exploration. Once the setup is complete, they initiate the testing process. CrashScope begins by dynamically identifying all UI components in the app, such as buttons, text fields, and other interactive elements. Using the LLM-powered exploration strategy, it examines labels, descriptions, and contextual information to infer expected behaviors and generate a sequence of inputs.

Reporting: The tool systematically interacts with the app, simulating taps, long presses, and entering valid and invalid data into text fields. It navigates through different screens, constructing a detailed transition graph that maps all the explored paths. During this process, if a crash occurs, CrashScope immediately records the incident. It takes a screenshot of the screen at the moment of the crash, logs the stack trace, and generates a step-by-step reproduction guide. Additionally, it creates a replayable script that allows the user to reproduce the crash on another device for further analysis.

LLM Summary: After the tests are completed, the user navigates to the crash report section of the web application. Here, they find a detailed report for each detected crash, which includes the stack trace, screenshots, reproduction steps, and replayable scripts. The LLM has also analyzed the crash data to generate a concise, human-readable summary of the incident, highlighting the key interactions that led to the crash.

4 Scenario Based Modeling

Scenario-based modeling is an approach used in software development and design to create models that depict how a system behaves or reacts in various real-world situations or scenarios. These scenarios are typically based on user interactions, events, or conditions and help developers understand and visualize how the software will function in different contexts. By simulating these scenarios, developers can better identify potential issues, make informed design decisions, and ensure that the software meets user requirements across different usage contexts.

4.1 Use Case Diagram

A use case diagram is a visual representation in software engineering that illustrates how a system interacts with its external entities or actors to achieve specific goals or tasks. It shows the relationships between use cases (representing system functionalities) and actors (representing external users or systems). Use case diagrams are typically created through requirements gathering, where system functionalities and user interactions are identified, and then diagrammed using specialized software modeling tools.

4.1.1 Use Case Level 0 – CrashScope LLM

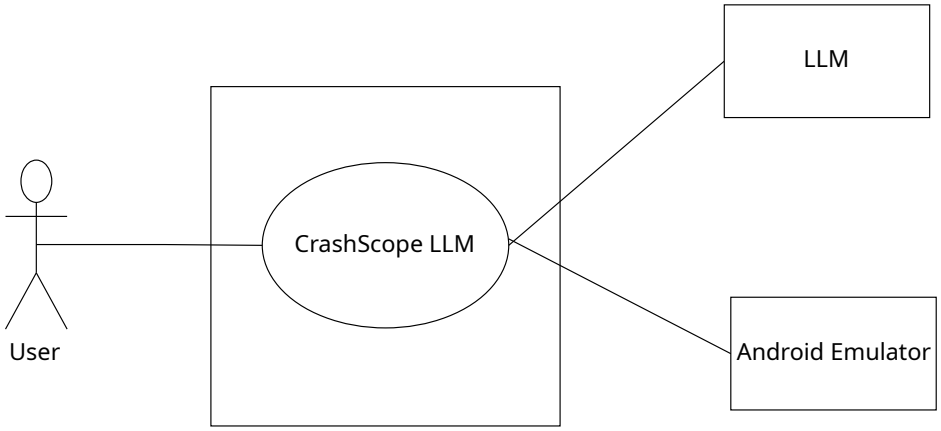


Figure 1: Use Case Level 0

Primary Actor: User

Secondary Actor: LLM, Android Emulator

Goal in Context: The diagram shown above represents the entire system of CrashScope LLM.

4.1.2 **Use Case Level 1 – Modules of CrashScope LLM**

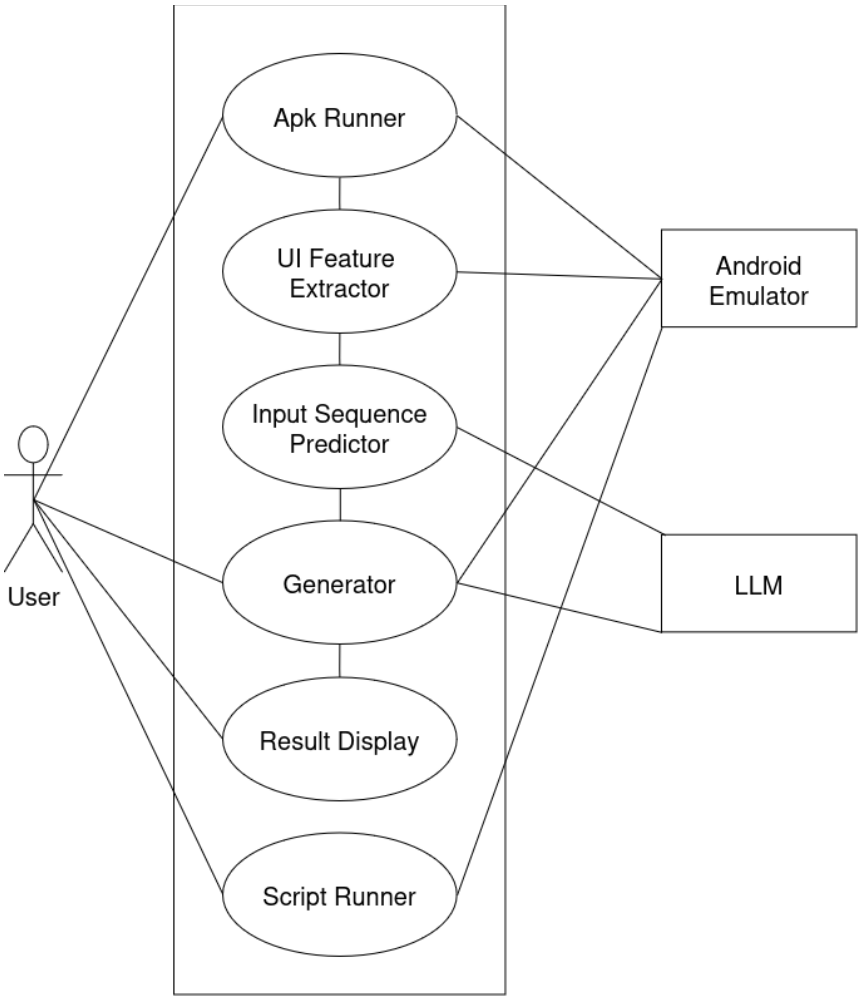


Figure 2: Use Case Level 1 - Modules of CrashScope LLM

Primary Actor: User
Secondary Actor: LLM, Android Emulator

Goal in Context: The diagram shows the modules of CrashScope LLM. CrashScope LLM consists of 5 modules. The modules can be elaborated as follows

4.1.2.1 ***Apk Runner***

Apk Runner runs the .apk in question. It uses Android Emulator to run the application. The module also communicates with Android Emulator. It gets the UI structure from the Emulator software. Then it sends the data to the next module for parsing. It also sends instructions to the Emulator to simulate various events such as tap, long tap, double tap etc.

4.1.2.2 ***UI Feature Extractor***

This module gets the structure from the Emulator and parses it. It then sends the data to the next module for deciding which node to visit.

4.1.2.3 ***Input Sequence Predictor***

Predicts the next input using the LLM module.

4.1.2.4 ***Generator***

Generates test cases, reports, and replayable scripts.

4.1.2.5 ***Result Display***

Shows how the app is running.

4.1.2.6 ***Script Runner***

Runs the test scripts generated by Crash Scope.

4.1.3 **Use Case Level 1.1 – Generator**

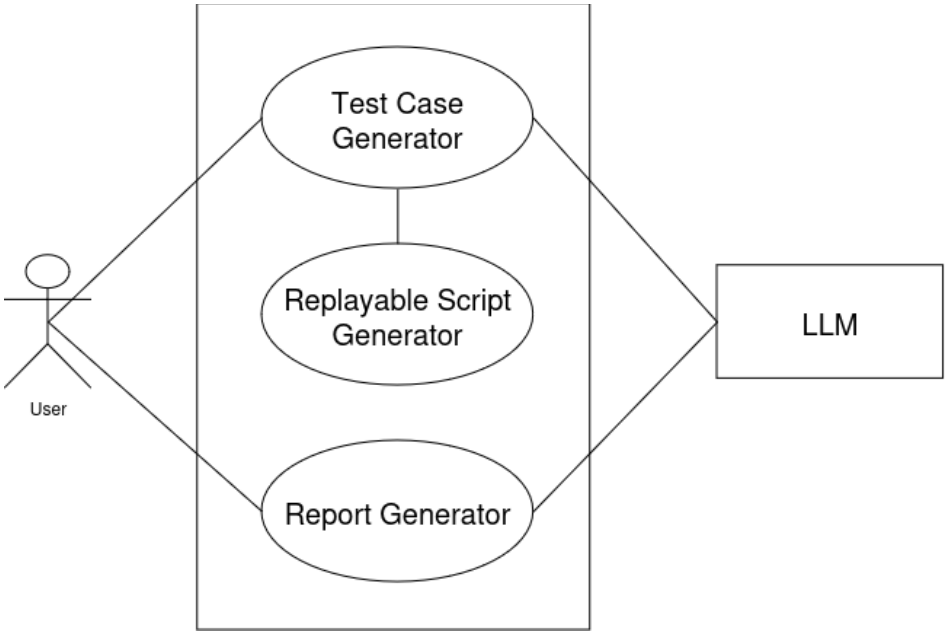


Figure 3: Use Case Level 1.1 - Generator

Primary Actor: User

Secondary Actor: LLM

5 Class-Based Modeling

Class-based modeling is a visual representation in software engineering that illustrates the structure of a software system using classes, their attributes, and relationships. It is commonly used in object-oriented design and modeling.

5.1 Analysis Classes

After identifying nouns from the scenario, I have filtered nouns belonging to the solution domain using General Classification (External entities, Things, Events, Roles, Organizational units, Places, and Structures). Nouns selected as a potential class were filtered using Selection Criteria (Retained information, Needed services, Multiple attributes, Common attributes, Common operations, and Essential requirements). After performing analysis on potential classes, I have found the following analysis classes -

- Runner
- Replayable Script Runner
- Simple Apk Runner
- Android Emulator
- CrashScope
- Feature Extractor
- Next Step Predictor
- Generator
- Test Case Generator
- Replayable Script Generator
- Crash Report Generator

After some careful consideration and knowledge gained from working on previous projects, I have decided to go with the following set of classes -

- CrashScope
- Apk Runner
- Script Runner
- Android Emulator
- Feature Extractor
- Test Case Generator
- Replayable Script Generator
- Report Generator

- LLM

5.2 Class Cards

The class cards are given below -

CrashScope	
Attributes	**Methods**
`-apkRunnerInstance` `-scriptRunnerInstance`	`+runApk()` `+runScript()`
Responsibilities	**Collaborators**
• Run the .apk files for UI exploration and generating test cases • Run the scripts to reproduce the crash	`ApkRunner, ScriptRunner`

Table 1: Class Card - CrashScope

Apk Runner	
Attributes	**Methods**
`-androidEmulatorInstance`	`+run()`
Responsibilities	**Collaborators**
• Runs the Emulator for exploring the UI	`AndroidEmulator`

Table 2: Class Card - ApkRunner

Replayable Script Runner	
Attributes	**Methods**
`-androidEmulatorInstance` `-script`	`+loadScript()` `+run()`
Responsibilities	**Collaborators**
• Run the scripts to reproduce the crash, or to perform tests	`AndroidEmulator`

Table 3: Class Card - Script Runner

Android Emulator	
Attributes	**Methods**
-featureExtractorInstance -reportGeneratorInstance	+getNextMove() +generateReports()
Responsibilities	**Collaborators**
• Get the next move from feature extractor • Generate reports for crashes, or in general, for any activity.	FeatureExtractor, ReportGenerator

Table 4: Class Card – Android Emulator

Feature Extractor	
Attributes	**Methods**
-tcGeneratorInstance -UIData -history	+setData() +analyzeFeatures() +getNextStep()
Responsibilities	**Collaborators**
• Analyze the UI, and parse it • Run the test case generator • Get the next node to explore	TestCaseGenerator

Table 5: Class Card – Feature Extractor

Report Generator	
Attributes	**Methods**
-data -report -summary -LLMInstance	+getReportSummary() +generateReport() +getCrashScript()
Responsibilities	**Collaborators**
• Generate crash reports • Summarize crash reports • Get crash scripts	LLM, ReplayabeScriptGenerator

Table 6: Class Card – Report Generator

Test Case Generator	
Attributes	**Methods**
-scriptGeneratorInstance -LLMInstance -data	+generateScripts()
Responsibilities	**Collaborators**
• GenerateTestScripts	ReplayableScriptGenerator, LLM

Table 7: Class Card – Test Case Generator

Large Language Model (LLM)	
Attributes	**Methods**
-history -summary -prompt	+getPredictions() +getSummary()
Responsibilities	**Collaborators**
• Get predictions • Get summary	

Table 8: Class Card – LLM

Replayable Script Generator	
Attributes	**Methods**
-data	+generateScript()
Responsibilities	**Collaborators**
• Generate the scripts to reproduce the crash, or to perform tests	

Table 9: Class Card - Script Runner

5.3 **CRC Diagram**

The Class Responsibility Collaborator diagram is given below -

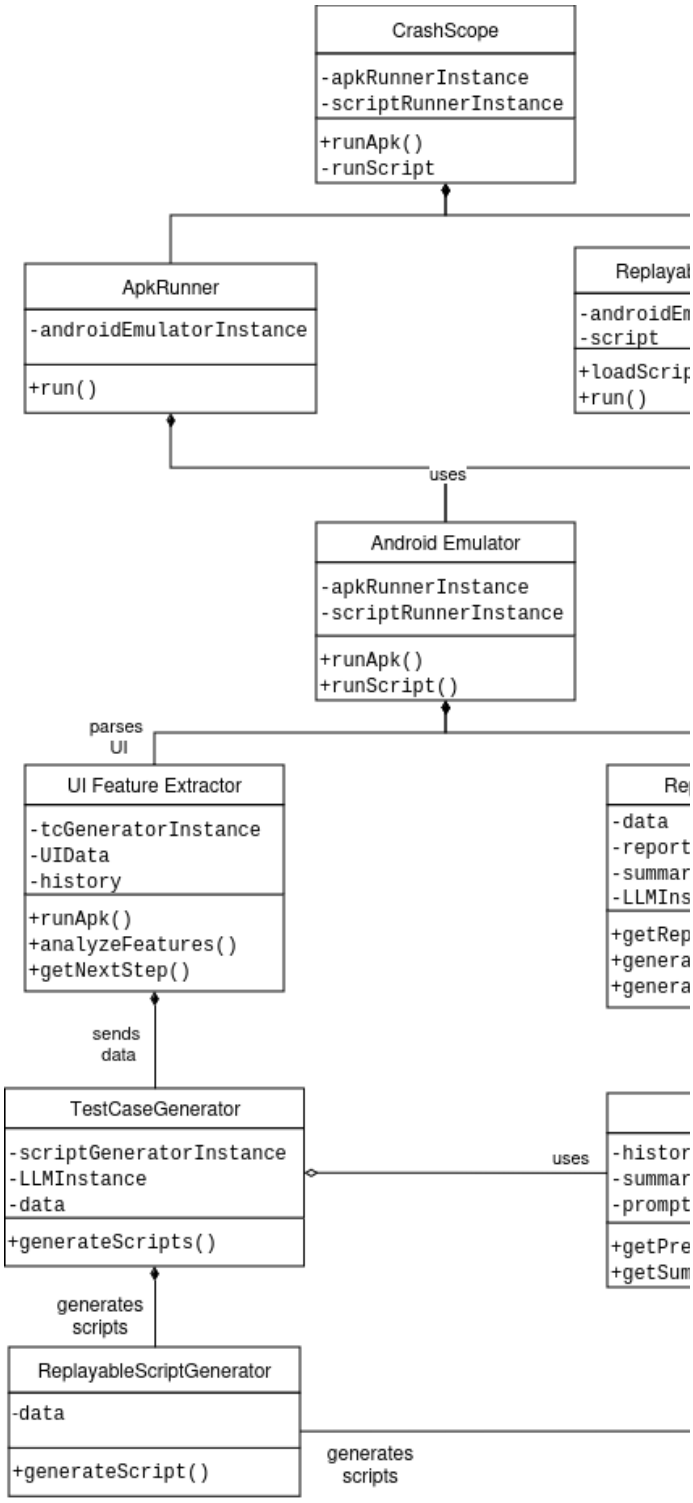


Figure 4: CRC Diagram

6 **Architectural Design**

Architectural design is a visual representation in software engineering that outlines the high-level structure and organization of a software system. It focuses on defining the major components or modules of the system, their interactions, and the overall system's architecture.

6.1 **Architectural Context Diagram**

The architectural context diagram for the tool is given below -

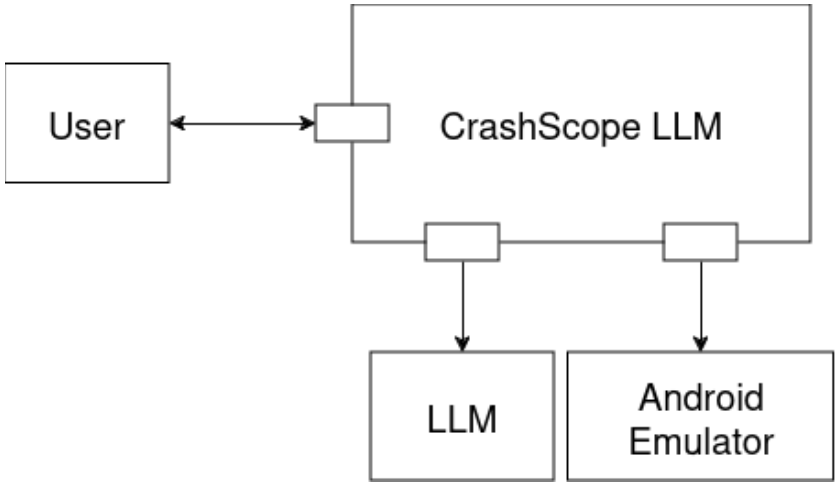


Figure 5: Architectural Context Diagram

Users can use the tool. They are producers and consumers of the information of the tool. There are two subordinate systems – LLM, and Android Emulator. LLM will run the necessary models, and the tool would get it's predictions, and summaries from the LLM. The tool will use the Android Emulator to run the apk file in the desktop.

6.2 **Top Level Components**

Overall architectural structure with top level components is illustrated below-

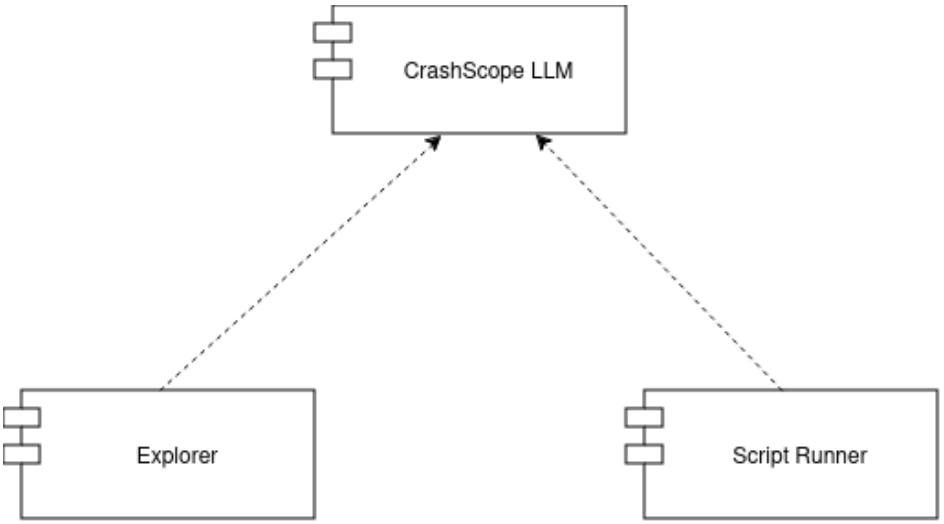


Figure 6: Top Level Components

6.3 **Instantiation of the Architecture**

The components identified in Section - 6.2 are broad. So they are refined and instantiated into smaller components. We use different architectural patterns which better suite each component.

6.3.1 **Explorer**

Pattern: Layered Architecture.

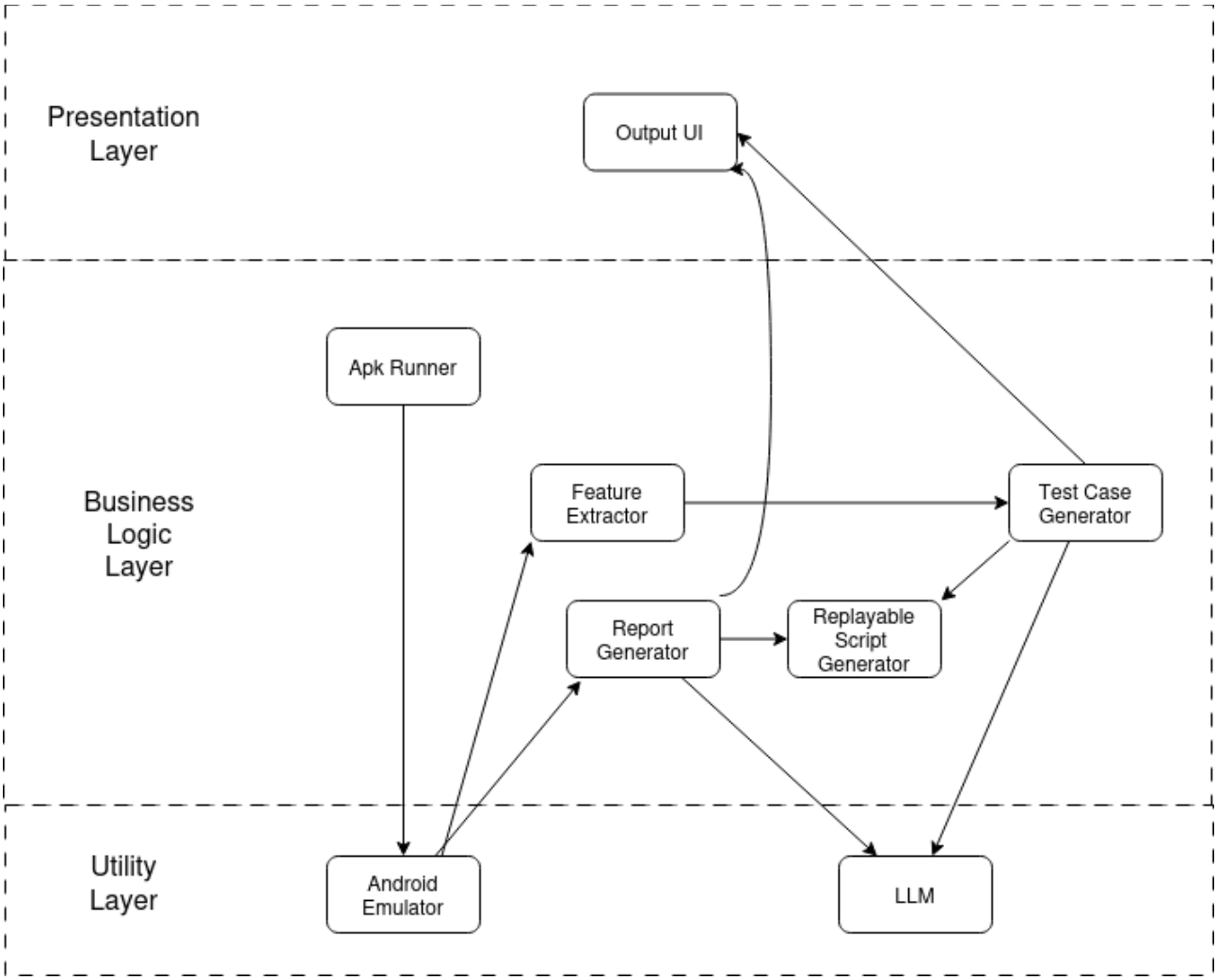


Figure 7: Layered Architecture for the Explorer Component

The explorer module explores the Android app. It uses LLMs to simulate a real-world scenario and which node to visit next. It also generates test cases accodingly.

6.3.2 **Script Runner**

Pattern: Layered Architecture

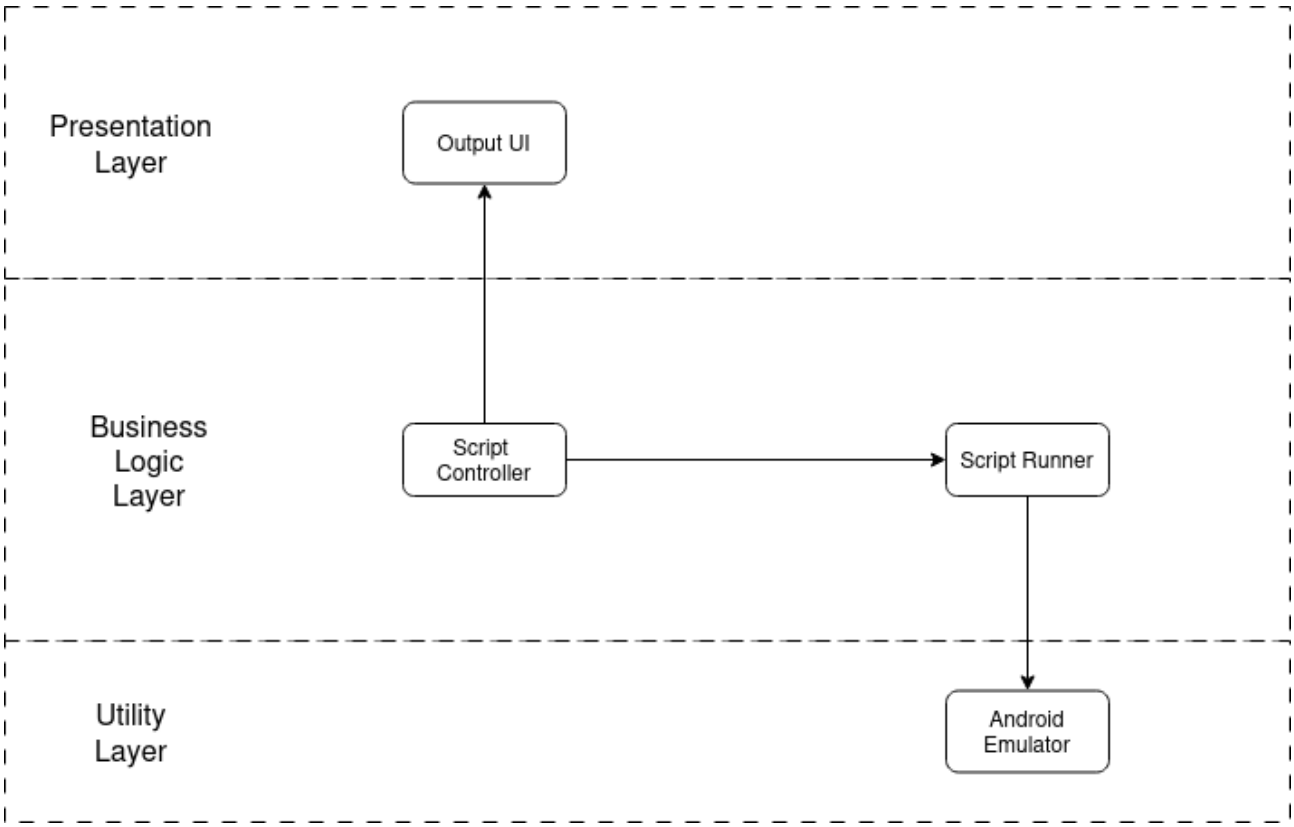


Figure 8: Layered Architecture for the Script Runner Component

This component runs the scripts to reproduce a crash, or to run the test cases.

7 Preliminary Test Plan

The primary objective is to test and validate the enhanced functionality of CrashScope integrated with an LLM. This includes the ability to explore Android application UIs effectively, generate test cases, and produce human-readable crash reports.

7.1 High Level Description of Testing Goals

- Validate the existing CrashScope features.
- Test new LLM-based exploration and reporting functionalities.
- Measure the response time and resource consumption during large-scale .apk testing.
- Validate individual functions such as UI exploration strategies and crash report generation.
- Test the interaction between CrashScope and the LLM.
- Perform end-to-end testing, including .apk input, crash detection, and report generation.
- Validate the final system against the proposed features.

7.2 Test Cases

Test ID: T1

Test Case: Verify that the LLM correctly identifies UI components and generates appropriate actions.

Steps To Be Executed:

1. Trigger a crash using a known input sequence.
2. Generate the crash report.

Input: .apk file

Expected Result: Report includes stack trace, screenshots, reproduction steps, and replayable scripts.

Pass/Fail: Pending

Test ID: T2

Test Case: Ensure users can select the desired UI exploration strategy.

Steps To Be Executed:

1. Select an exploration strategy (Top-down, Bottom-up, Random, LLM).
2. Execute the exploration.

Input: .apk file

Expected Result: Selected strategy is executed correctly.

Pass/Fail: Pending

Test ID: T3

Test Case: Validate the crash report's completeness and accuracy.

Steps To Be Executed:

1. Trigger a crash using a known input sequence.
2. Generate the crash report.

Input: .apk file

Expected Result: Report includes stack trace, screenshots, reproduction steps, and replayable scripts.

Pass/Fail: Pending

Test ID: T4

Test Case: Validate that testing works for diverse .apk files.

Steps To Be Executed:

1. Provide multiple .apk files with varying complexities.
2. Test each file.

Input: .apk file

Expected Result: All .apk files are tested without errors.

Pass/Fail: Pending

Test ID: T5

Test Case: Validate the parameter configuration page.

Steps To Be Executed:

1. Open the application.
2. Configure parameters for testing.

Input: .apk file, text

Expected Result: Parameters are saved and applied during testing.

Pass/Fail: Pending

Test ID: T6

Test Case: Ensure crash reports are displayed correctly.

Steps To Be Executed:

1. Trigger a crash.
2. View the generated report in the web app.

Input: .apk file

Expected Result: Report is readable and contains all necessary information.

Pass/Fail: Pending

Test ID: T7

Test Case: Ensure clickable elements (e.g., buttons, checkboxes) are correctly identified.

Steps To Be Executed:

1. Input an .apk file with known clickable elements.
2. Run CrashScope with the LLM exploration strategy.

Input: .apk file

Expected Result: All clickable elements are identified and stored in a stack.

Pass/Fail: Pending

Test ID: T8

Test Case: Verify that text input fields are detected and processed correctly.

Steps To Be Executed:

1. Input an .apk file with email, number, and text fields.
2. Trigger LLM to enter valid and invalid data into fields.

Input: .apk file

Expected Result: Inputs are correctly identified and crashes are triggered for invalid inputs.

Pass/Fail: Pending

Test ID: T9

Test Case: Validate that replayable scripts reproduce crashes.

Steps To Be Executed:

1. Trigger a crash.
2. Execute the replayable script on another device.

Input: .apk file

Expected Result: The crash is reproduced successfully.

Pass/Fail: Pending

Test ID: T10

Test Case: Validate the LLM's ability to use contextual information for decision-making.

Steps To Be Executed:

1. Provide an .apk file with complex navigation and contextual UI labels.

Input: .apk file

Expected Result: LLM correctly infers expected behaviors and generates meaningful test cases.

Pass/Fail: Pending

8 Conclusion

The integration of an LLM-powered CrashScope redefines how developers approach automated testing for Android applications. This tool eliminates the need for manual testing and complex instrumentation, making crash detection accessible and efficient. By leveraging advanced AI, CrashScope intelligently explores an app's UI, simulating real-world interactions and identifying vulnerabilities that traditional tools might miss. The inclusion of human-readable crash reports and reproducible scripts empowers developers to debug efficiently and address critical issues with precision.

For developers, this tool is indispensable in ensuring application stability and delivering high-quality user experiences. With the increasing complexity of modern apps and the demand for rapid development cycles, CrashScope reduces testing time, enhances accuracy, and provides actionable insights. It not only identifies crashes but also helps developers understand their root causes in a concise, intuitive manner.

By adopting this enhanced CrashScope, developers can confidently release applications, knowing they are robust and well-tested. It bridges the gap between sophisticated crash analysis and usability, offering a seamless, efficient, and intelligent solution to one of the most challenging aspects of software development.

9 References

[1] Moran, Kevin, et al. ‘CrashScope: A Practical Tool for Automated Testing of Android Applications’. 2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C), IEEE, 2017, https://doi.org/10.1109/icse-c.2017.16.

[2] Moran, Kevin, et al. ‘Automatically Discovering, Reporting and Reproducing Android Application Crashes’. *2016 IEEE International Conference on Software Testing, Verification and Validation (ICST)*, IEEE, 2016, https://doi.org/10.1109/icst.2016.34.